

Deep Learning

by: Ahmad Masarrat
msratahmad@gmail.com

Outline

Introduction to DL

NN Fundamentals

DNN Architectures

Optimization & Cost Functions

Deep & Practical Concepts

NoteBooks

What is deep learning?

Key Components

Neurons

**Neural
Networks**

**Loss
Function**

Optimizer

Activation Function

Structure of a Neuron

Inputs ($x_1, x_2, \dots, x_n$)

Weights ($w_1, w_2, \dots, w_n$)

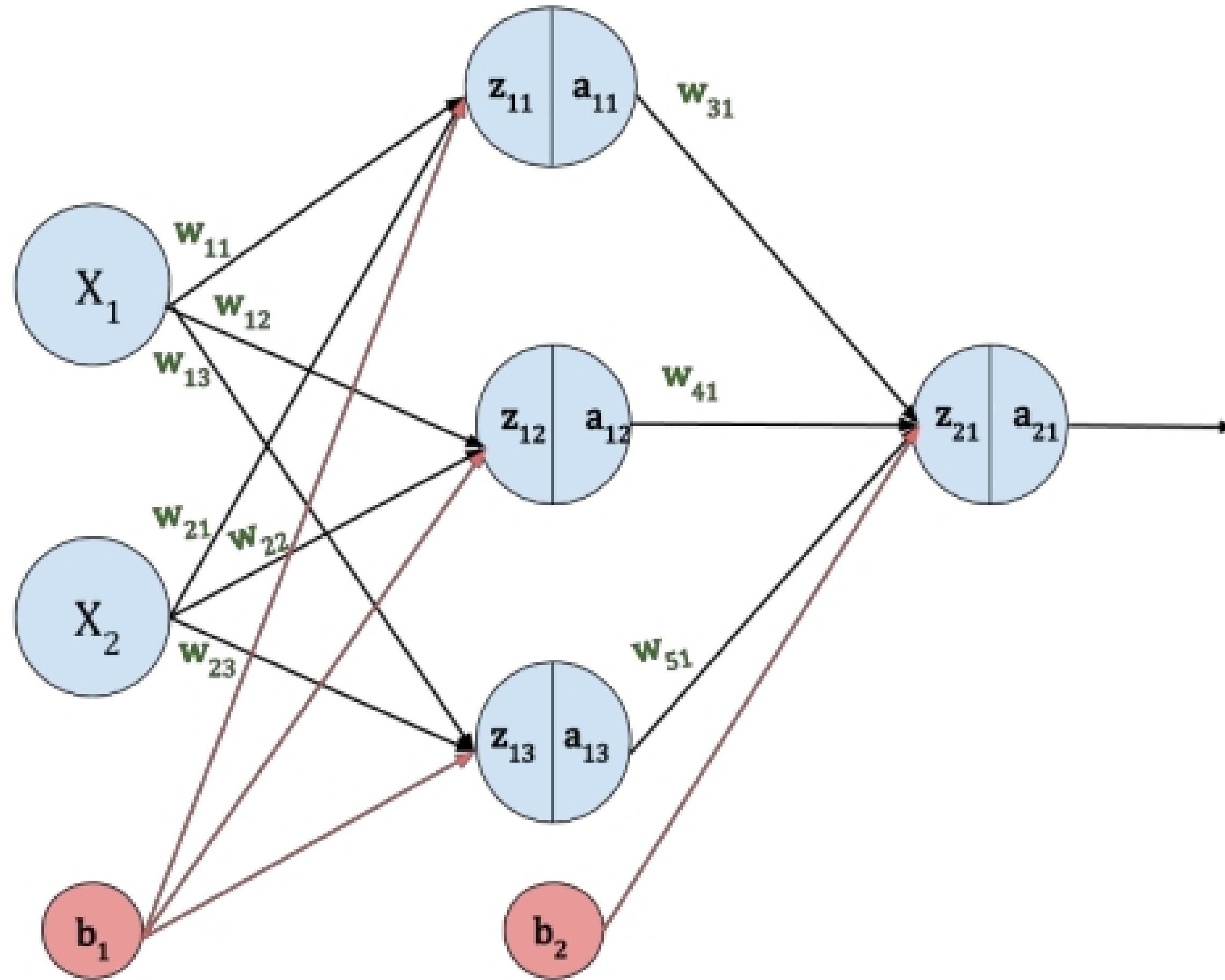
Bias (b)

Weighted Sum (z)

$$z = (w_1 \cdot x_1) + (w_2 \cdot x_2) + \dots + (w_n \cdot x_n) + b$$

z is passed through an **Activation Function**

Structure of a Neuron



Loss Functions

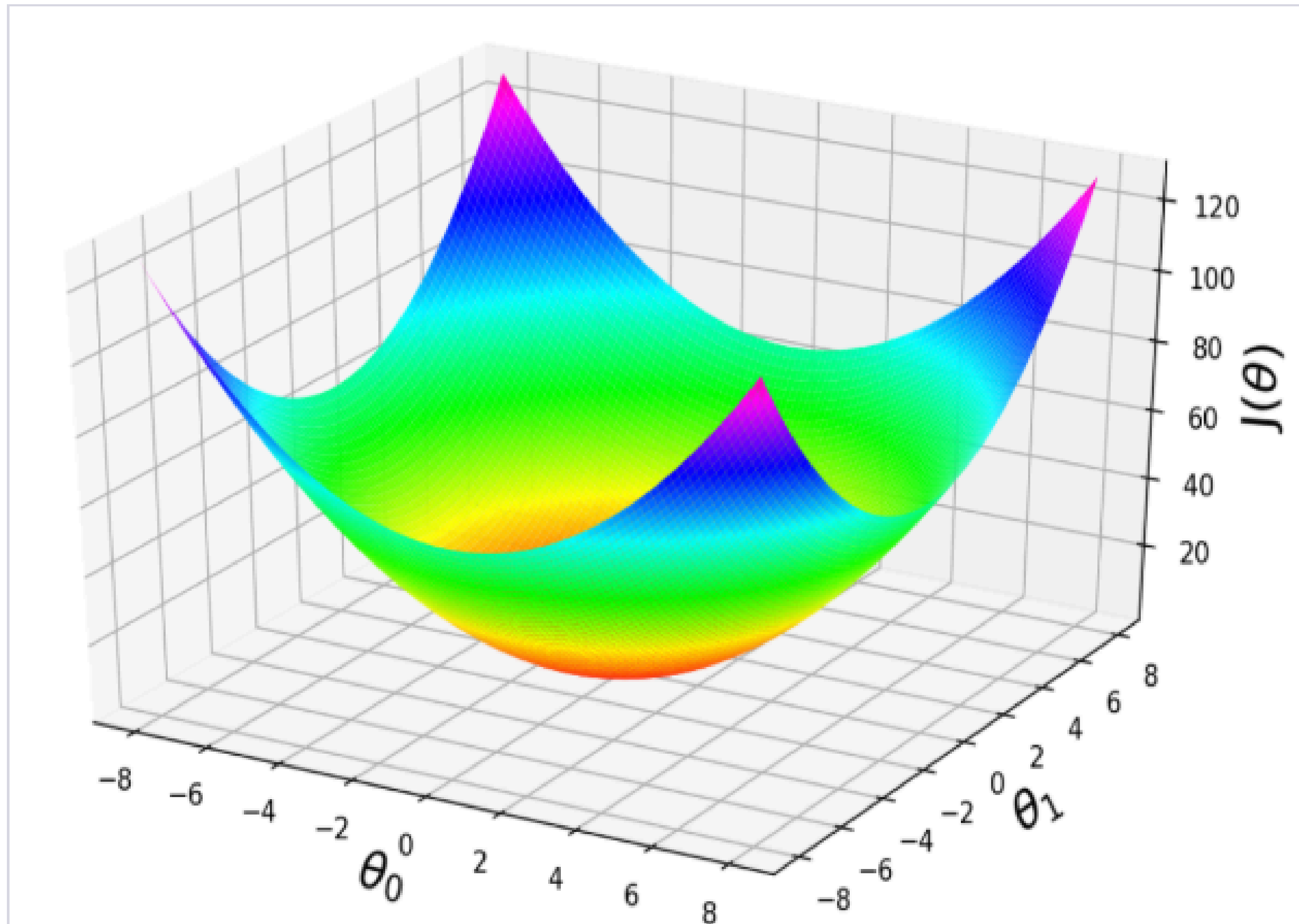
- Mean Squared Error (MSE) for regression.

measures distance

- Cross-Entropy Loss for classification problems.

measures probability

Backpropagation and GD



Neural networks **without** an activation function

hidden layer calculation

$$z_{h_1} = w_1 i_1 + w_2 i_2 + b_1$$

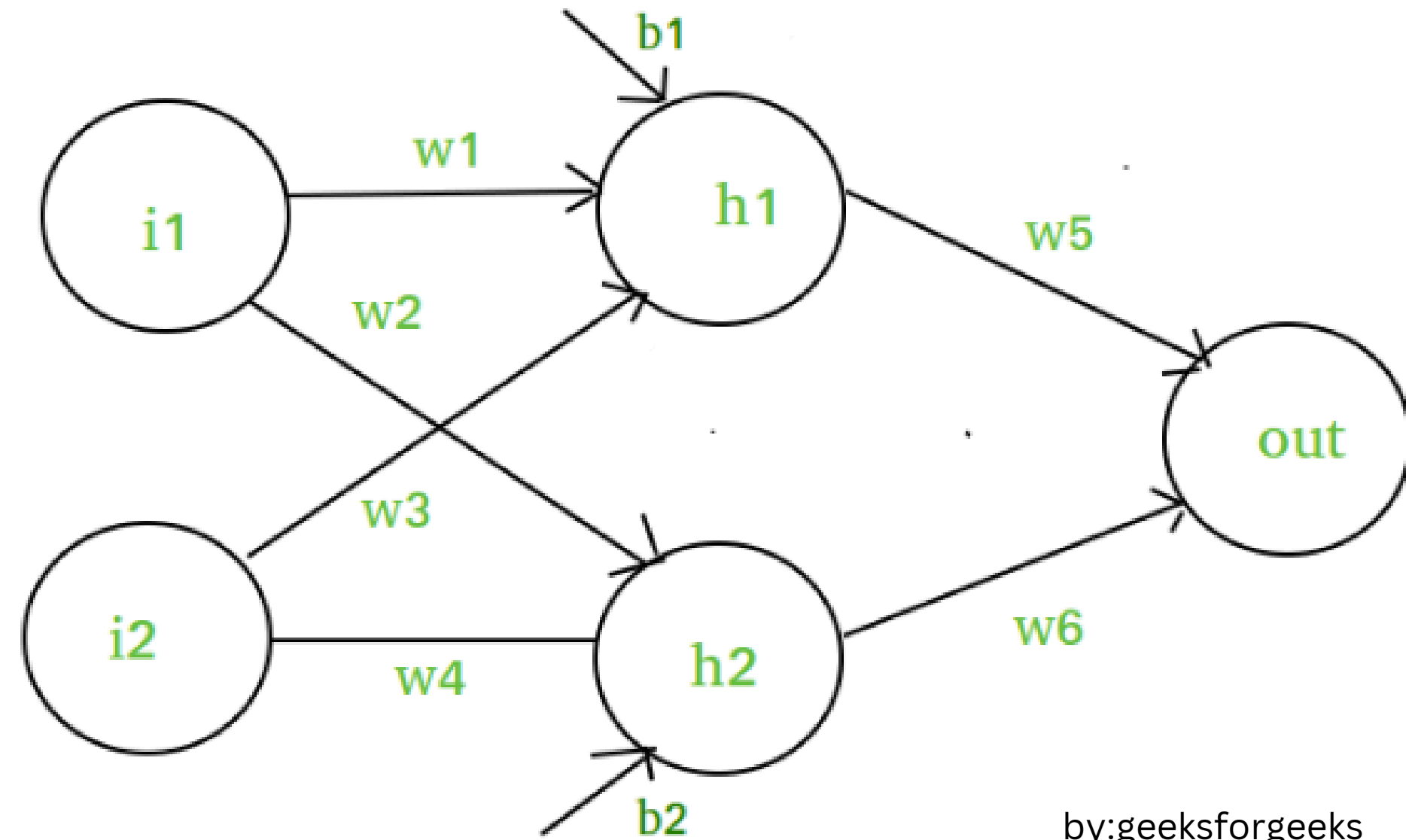
Output Layer Calculation

$$\text{output} = w_5 h_1 + b_2$$

**If h1 were directly the output of zh1(bc
no activation func)**

$$\text{output} = w_5(w_1 i_1 + w_2 i_2 + b_1) + b_2$$

$$\text{output} = w_5 w_1 i_1 + w_5 w_2 i_2 + w_5 b_1 + b_2$$



by:geeksforgeeks

Neural networks **with** an activation function

activation function: ReLU

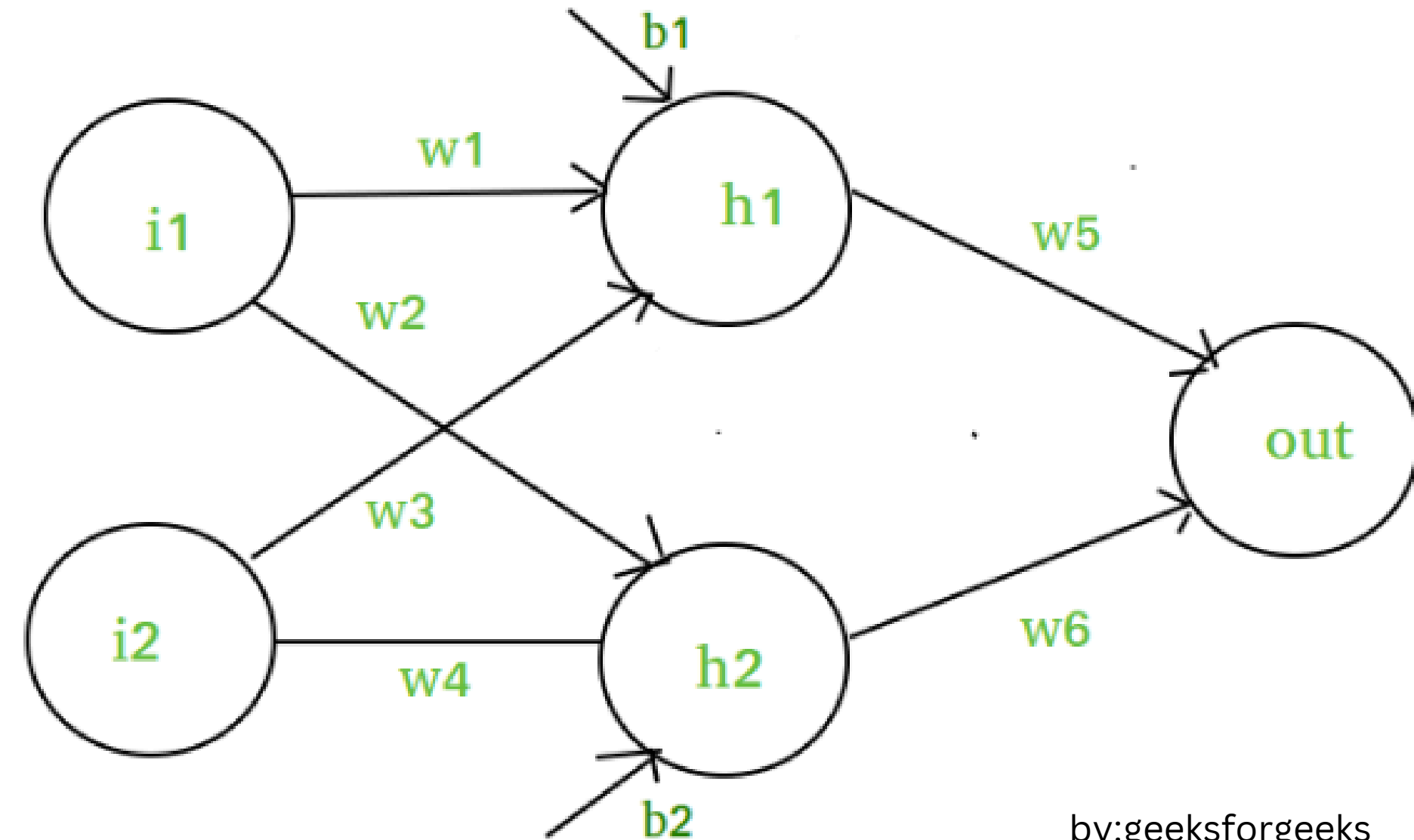
$$\sigma(x) = \max(0, x).$$

hidden layer calculation

$$h_1 = \sigma(z_{h_1}) = \sigma(w_1 i_1 + w_2 i_2 + b_1)$$

Output Layer Calculation

$$\text{output} = w_5 \sigma(w_1 i_1 + w_2 i_2 + b_1) + b_2$$

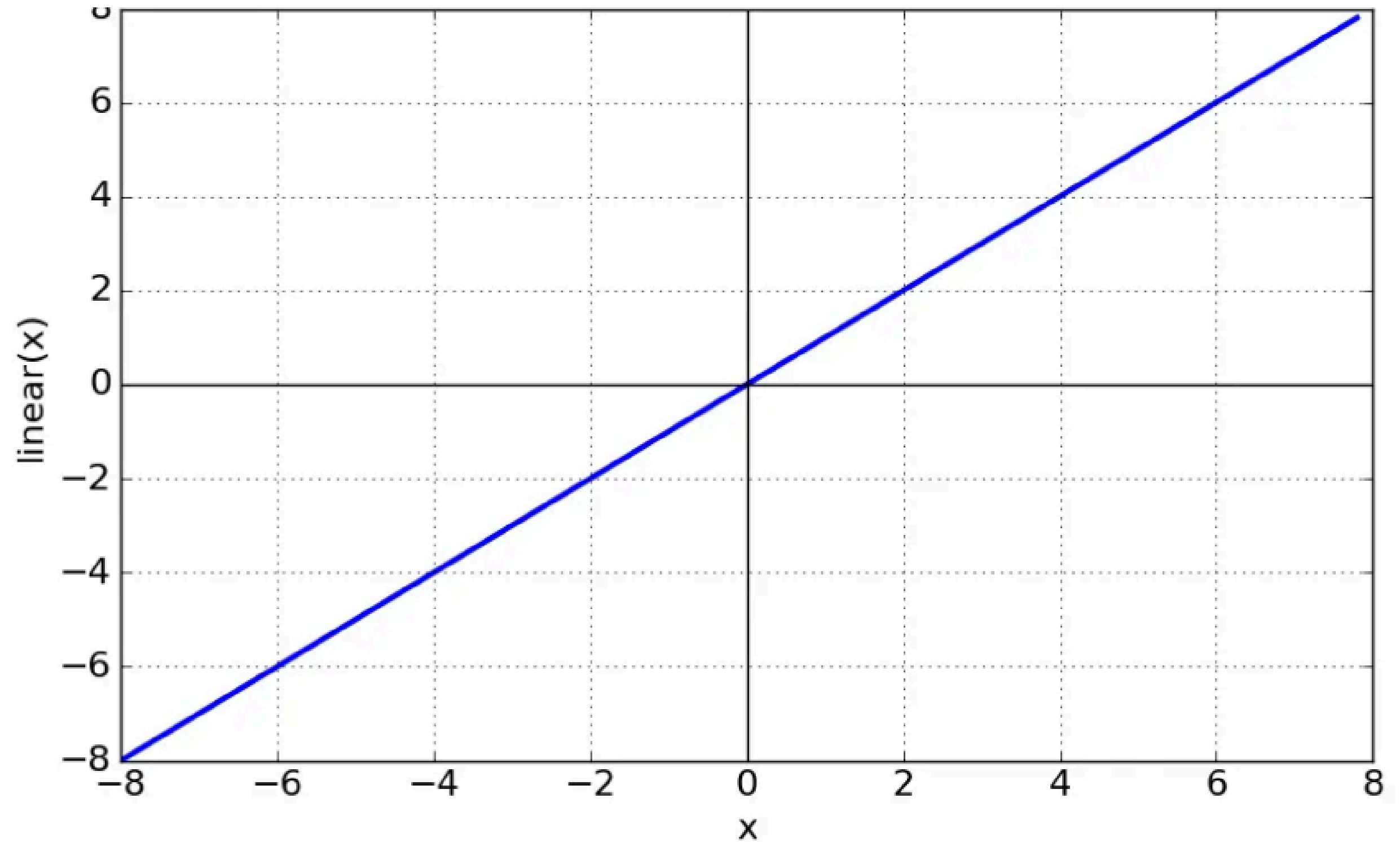


by:geeksforgeeks

Activation Functions

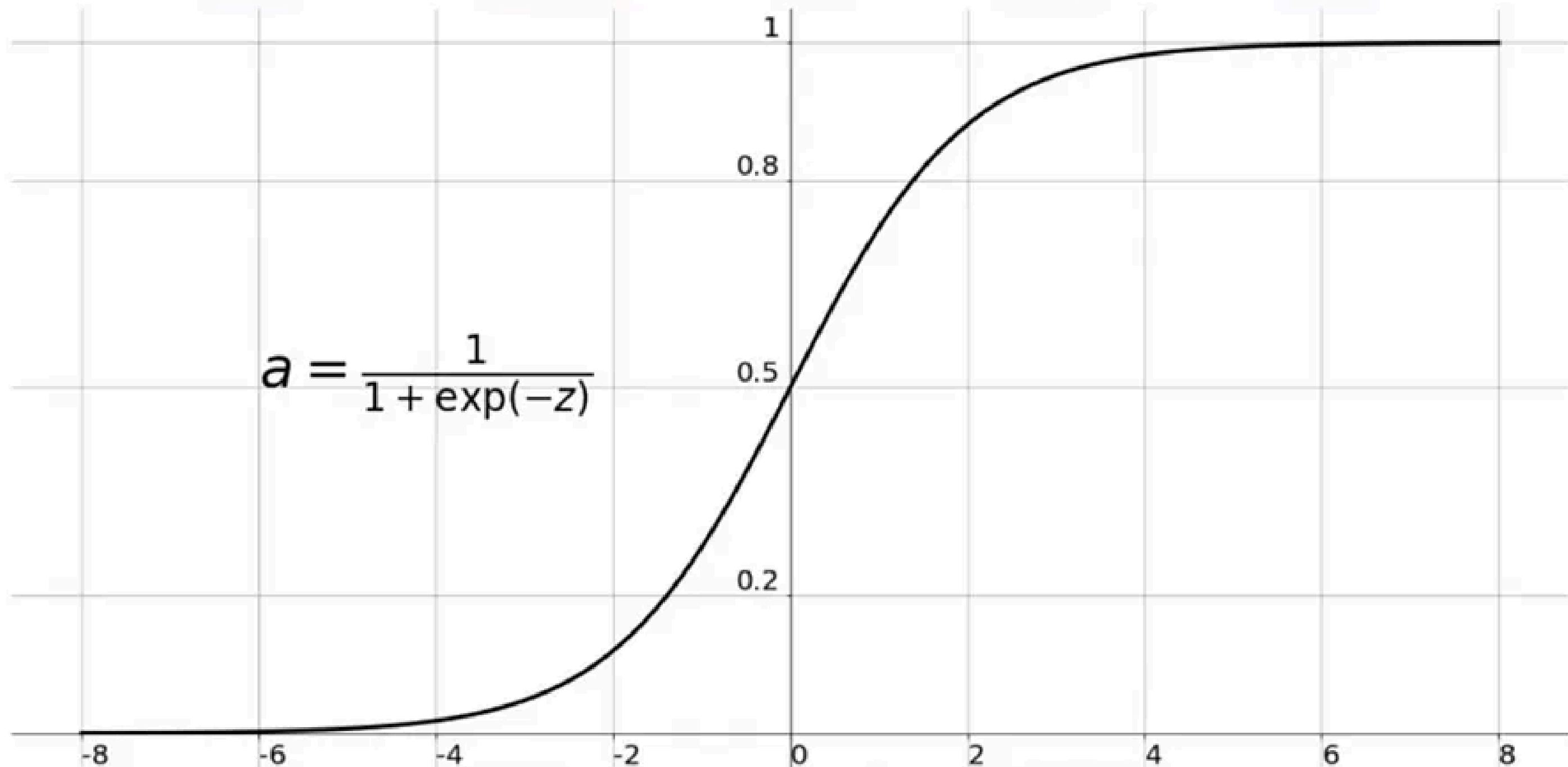
Linear Function

- used at just one place



Activation Functions

Sigmoid

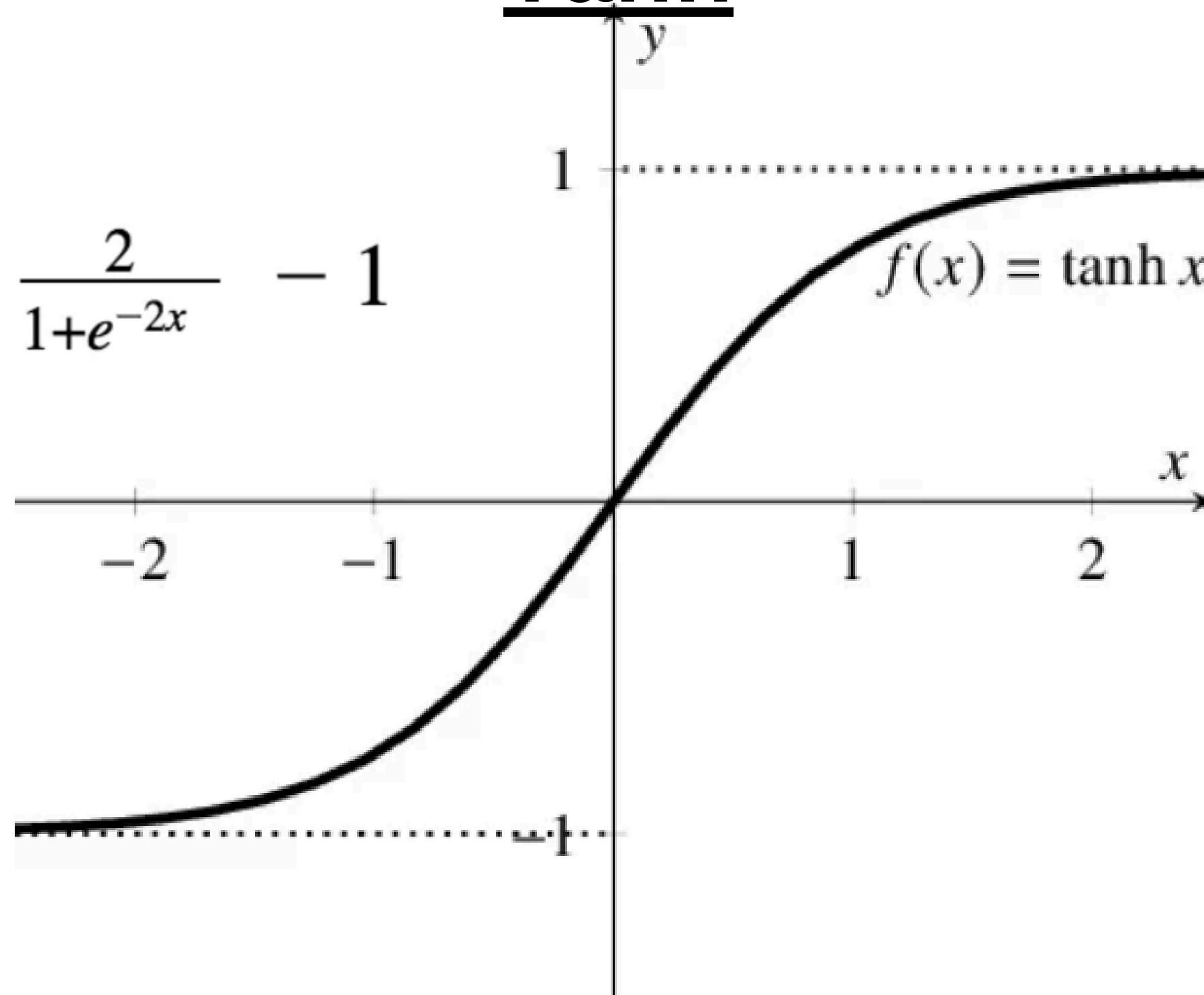


Activation Functions

Tanh

sigmoid

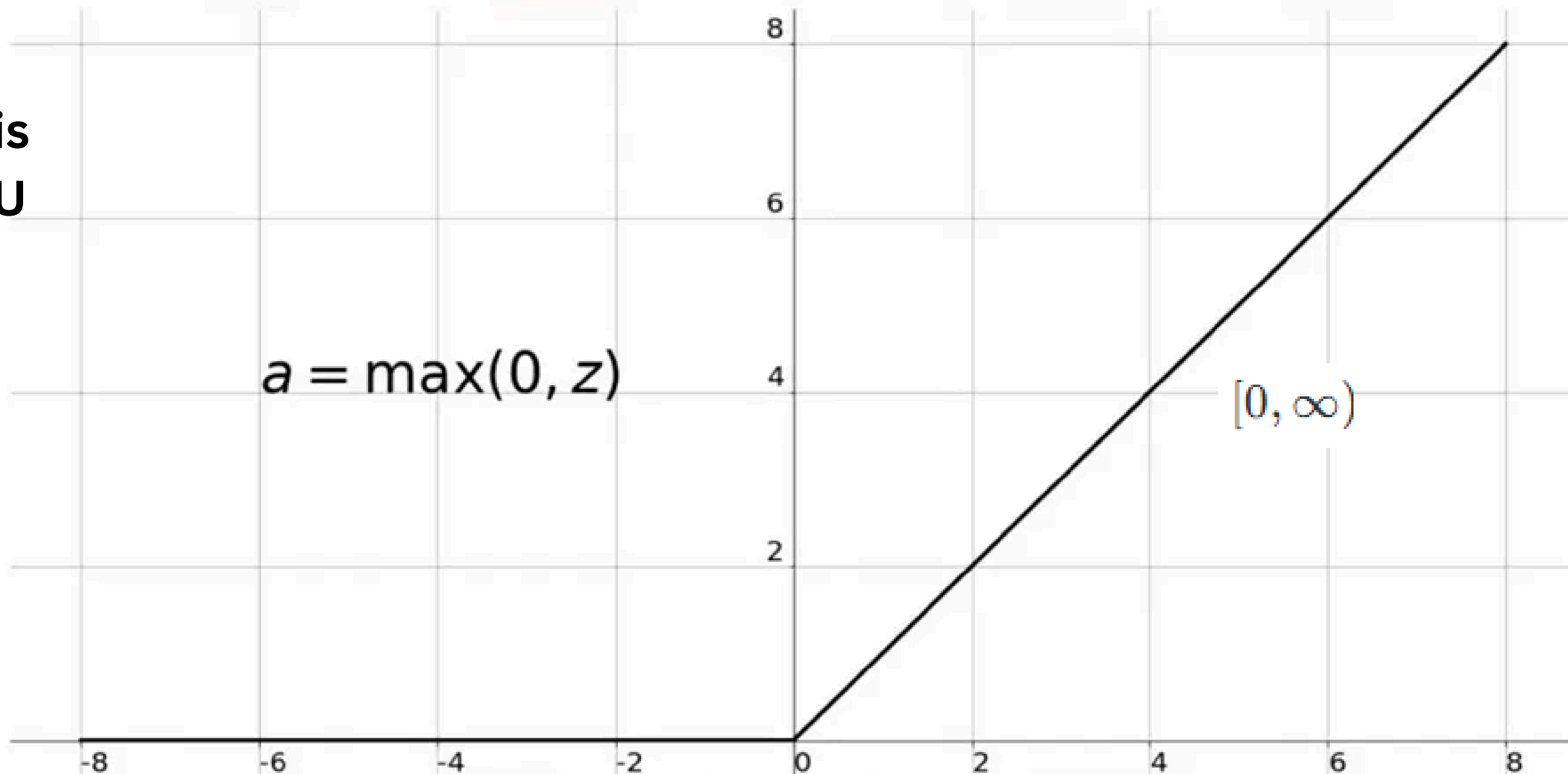
major advantage is
that the negative
inputs will be
mapped strongly
negative and the zero
inputs will be
mapped near zero



Activation Functions

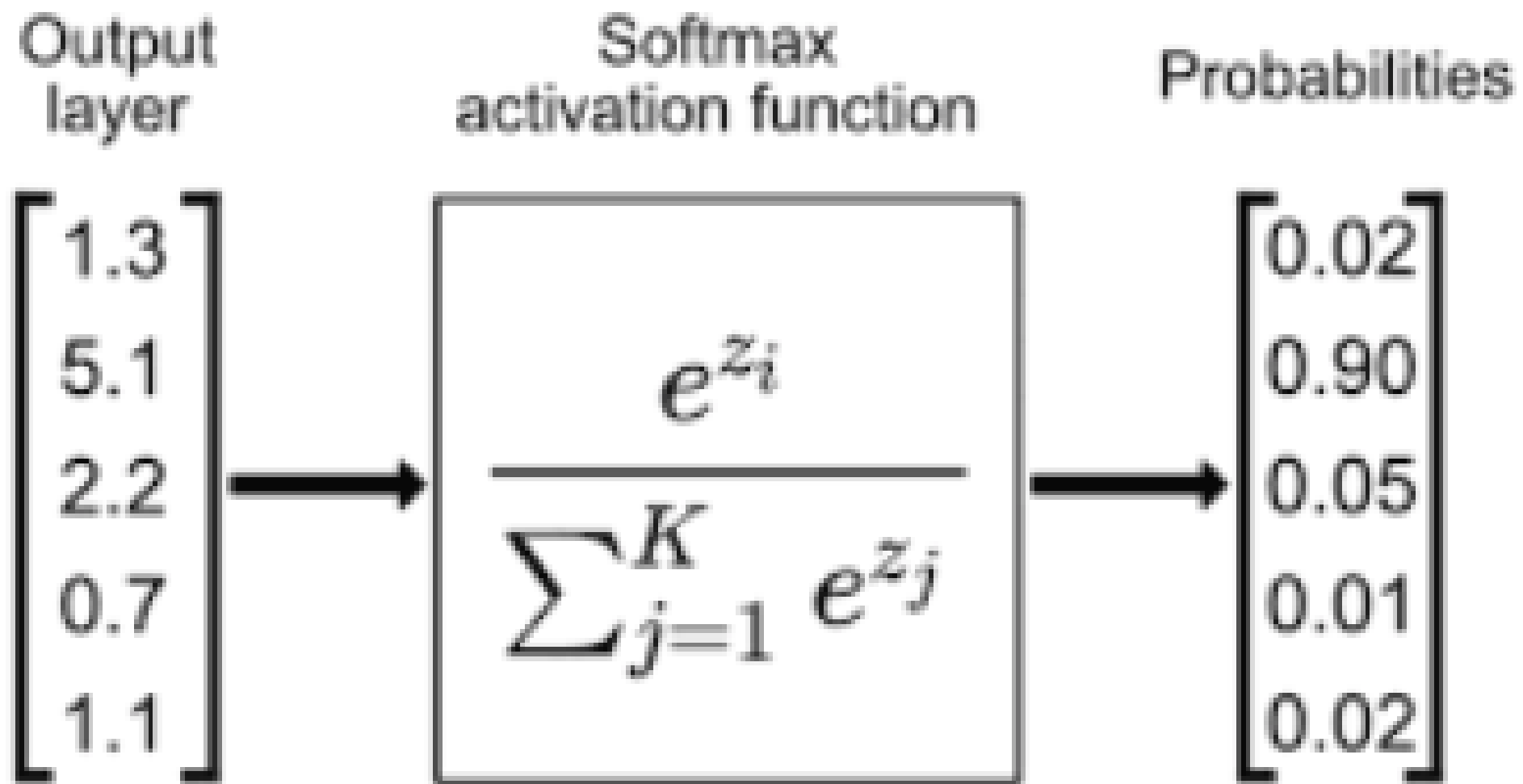
ReLU (Rectified Linear Unit)

if the input z is positive, ReLU returns x , if the input is negative, it returns 0



Activation Functions

Softmax



Softplus

$$A(x) = \log(1 + e^x)$$

the output is
always positive and
differentiable at all
points

Training Neural Networks

**Gradient Descent
and SGD**

Learning Rate (LR)

Optimizers

Regularization And Generalization

smarter model = higher risk of overfitting

how to avoid it ?

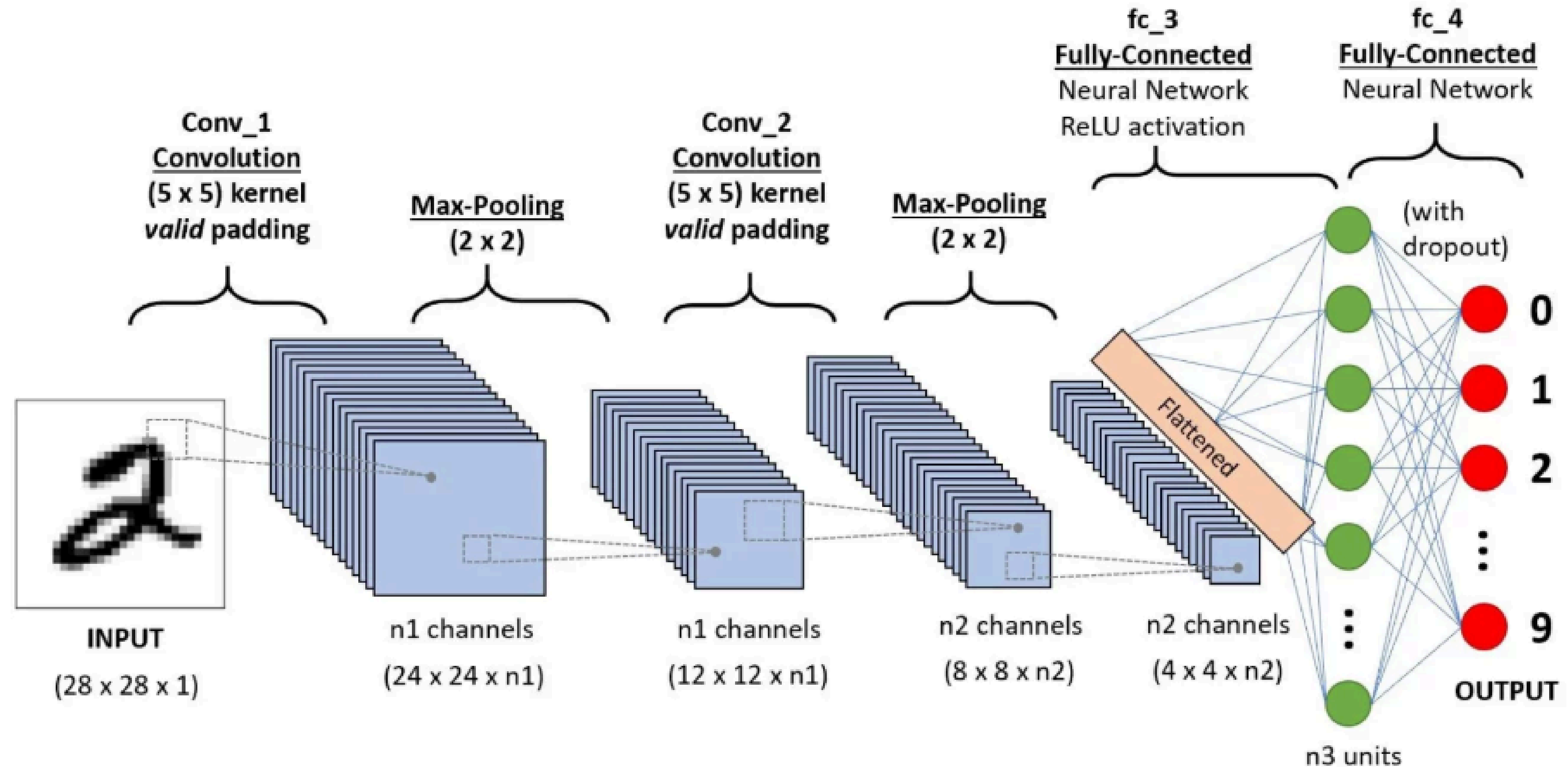
L2 / Weight Decay

Dropout

Batch
Normalization

Data Augmentation

Advanced Architectures



2	4	9	1	4
2	1	4	4	6
1	1	2	9	2
7	3	5	1	3
2	3	4	8	5

Image

x

1	2	3
-4	7	4
2	-5	1

Filter /
Kernel

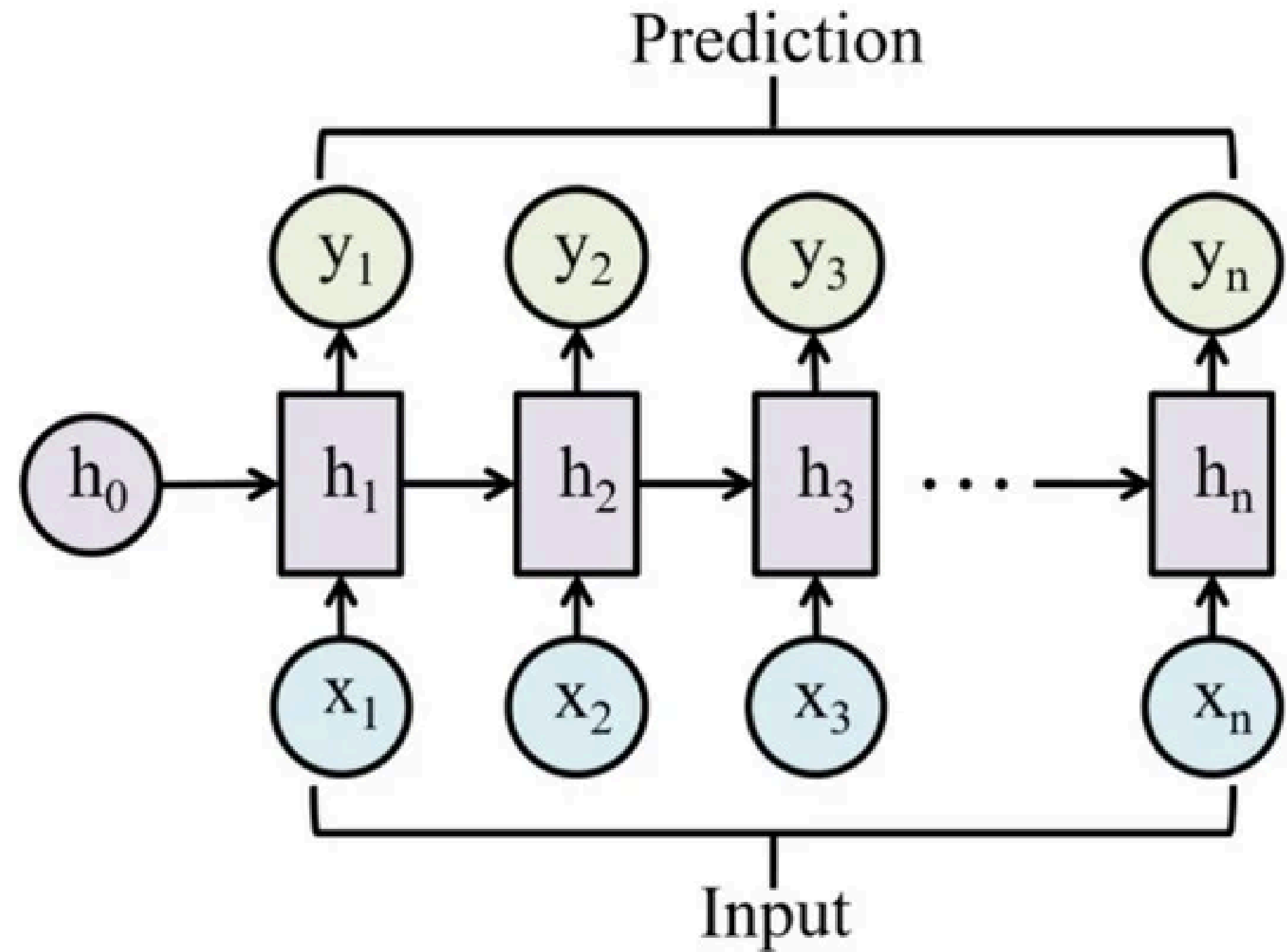
=

51		

Feature

pooling | padding | striding

Recurrent NN



LSTM/GRU



the gate is like a control unit

Deeper Topics in Training and Optimization

Initialization

Xavier/Glorot or
He Initialization

Residual Networks (ResNets)

skip

Training Tricks

LR Schedules
Early Stopping
Gradient Clipping

Practical Aspects of Deep Learning

GPU

or if u can wait for
forever, use the
CPU

Popular frameworks

PyTorch
TensorFlow

Practical Aspects of Deep Learning

Data Pipeline and Datasets

Data Cleansing

outliers
corrupted
missing

Data Splits

Train
validation
test

Batching

mini ones

Practical Aspects of Deep Learning

Evaluation Metrics

Classification

Accuracy,
Precision/Recall,
F1-Score,
ROC
AUC.

Regression

MSE, MAE, R^2 .

Ranking

MAP, NDCG.

Practical Aspects of Deep Learning

Hyperparameter Tuning

Hyperparameters

LR
batch size
layers number
hidden units
dropout rate
etc

Search Methods

Grid search
Random Search
Bayesian Optimization

Start small

computation is not
for free

Notebooks

Fashion MNIST dataset

